

Vade Mecum User Guide

1. Getting started	2
1.1. Setting up	2
1.2. Features	2
2. Available settings	3
2.1. lang	3
2.2. font	3
2.3. font-size	3
2.4. heading-numbering	3
2.5. page-numbering	3
2.6. page-number-alignment	3
2.7. bullet-point-indent	3
2.8. use-hydra	4
2.9. hydra-margins	4
2.10. justify	4
2.11. code-font	4
2.12. code-font-size	4
2.13. code-bg-color	4
2.14. codly-config	4
2.15. syntax-theme	5
2.16. math-font	5
2.17. math-counter-levels	5
2.18. math-numbering	5
3. Utilities	6
3.1. Functions	6
3.1.1. code-block	6
3.1.2. float_table_from_2d-array	6
3.1.3. int_round	7
3.1.4. is-content-empty	8
3.1.5. is-content-non-empty	8
3.1.6. table_from_columns	8
3.2. Custom frame utilities	10
3.2.1. hidden	10
3.2.2. hidden-no-title	10
3.2.3. hidden-sup-only	10
3.2.4. pre-post-style	11
3.2.5. setup-frame-environment	11
3.3. Math symbols	12
4. Environments	13
4.1. Math Environments	13
4.5. Custom Environments	16

1. Getting started

1.1. Setting up

To get started, you can use the following snippet in your document:

```
1  #import "@local/vade-mecum:0.1.0": *
2
3  #show: e.set_(
4      settings,
5      // Configuration options are set here
6      font: "Fira Sans",
7      heading-numbering: "1.",
8      code-font-size: 12pt,
9      justify: false,
10     // etc.
11 )
12
13 #show: template
```

typ

1.2. Features

This package is a personal all-in-one Typst template, with the following features:

- Configurable basic settings. See [Section 2](#) for a list of available settings.
- Opinionated Hydra integration for section headings in page headers.
- Styling for certain Typst elements, including links, outlines, math symbols and references.
- Includes [Itemize](#)'s list/enum fixes and referencing (which is [still not possible in Typst](#))
- Includes [big-todo](#) for **TODO: notes** in the document.
- Uses [Codly](#) with opinionated defaults for code blocks.
- A bunch of utility functions. See [Section 3](#) for a list of available functions.
- A bunch of shorthand math symbols. See [Section 3.3](#) for a list of available symbols.
- Environments for math use cases, using [frame-it](#) See [Section 4.1](#) for more information.
- Utilities for building custom [frame-it](#) frames. See [Section 4.5](#) for an example, and see [Section 3.2](#) for the reference.

2. Available settings

lang

Type: string

Overall language of the document. This is used by Typst to determine hyphenation and other language-specific settings.

Default value: en

font

Type: string

Main font of the document. Does not apply to Math mode nor Raw mode.

Default value: Linux Libertine O

font-size

Type: length

Font size for the main document.

Default value: 12pt

heading-numbering

Type: function or string

Heading numbering pattern. Uses same pattern as Typst's numbering function.

Default value: 1.1.

page-numbering

Type: function or string

Page numbering pattern. Uses same pattern as Typst's numbering function.

Default value: 1

page-number-alignment

Type: alignment

Alignment of page numbers in the page footer.

Default value: center

bullet-point-indent

Type: length

Default indentation for Typst's enumerations and lists.

Default value: 1em

2. Available settings

`use-hydra`

Type: boolean

Whether to use Hydra for section headings in page headers. By default, the first level heading is shown at the left of the page header, and the second level heading is shown at the right of the page header. Note that this option will force each section to start on a new page, and that Hydra won't show in the first page of each section.

Default value: `false`

`hydra-margins`

Type: alignment or dictionary

Adapted page margins used when Hydra is enabled. This is mostly useful for increasing the top margin to make more room for the header.

Default value: (top: `8em`)

`justify`

Type: boolean

Whether to justify text or not.

Default value: `true`

`code-font`

Type: string

Font used for code blocks.

Default value: FiraCode Nerd Font

`code-font-size`

Type: length

Font size used for code blocks.

Default value: `9pt`

`code-bg-color`

Type: color

Background color for code blocks.

Default value: `rgb("#f6f8fa")`

`codly-config`

Type: arguments

Default configuration arguments for Codly code blocks.

Default value: `arguments(zebra-fill: none)`

`syntax-theme`

Type: path

Syntax theme for code blocks. This should be a path to a .tmtheme file.

Default value: `path("/src/themes/Github Light.tmtheme")`

`math-font`

Type: string

Font used for mathematical content.

Default value: Erewhon Math

`math-counter-levels`

Type: integer

Levels of numbering dependence for math frames. This determines how many levels of numbering depend on the section hierarchy. For example, if this is set to 2, the first two counters in the numbering pattern of math frames will use the section counters, while the rest will increment independently. This last counter will be reset at the beginning of each section.

Default value: 0

`math-numbering`

Type: none or string

Numbering pattern for math frames. Uses the same pattern as Typst's numbering function.

Default value: 1.1

3. Utilities

This section acts as a reference for the utility functions provided by this package.

3.1. Functions

3.1.1. code-block

Returns a code block with the contents of a file, using the filename as the header and its extension as the language for syntax highlighting. The code block is breakable, allowing it to span multiple pages if necessary.

NOTE: The language detection based on the file extension currently does NOT work. Prior to Typst 0.15, paths did not exist and thus the user had to pass the raw contents of the file to this function, along with the file name. Now, path support exists, but no operations are allowed with them, including extracting their string representation. For now, the extension will have to be manually specified

Parameters

```
code-block(  
  filename: path ,  
  lang: str  
) -> content
```

filename path

The path to the file to read.

lang str

Extension of the file to use for syntax highlighting. If empty, no syntax highlighting will be applied.

Default: ""

3.1.2. float_table_from_2d-array

Creates a table from a 2D array of data, with optional headers for the rows and columns.

Parameters

```
float_table_from_2d-array(  
  header_up: array<content> string> ,  
  header_side: array<content> string> ,  
  data: array<content> ,  
  kwargs: arguments (none) ,  
  precision: int  
) -> content
```

header_up `array<content>` or `string`

The header for the columns (top row).

header_side `array<content>` or `string`

The header for the rows (first column).

data `array<content>`

The 2D array of data to populate the table.

kwargs `arguments` or `none`

Additional keyword arguments to pass to the table function. If `none`, defaults to center alignment and equal column widths.

Default: `none`

precision `int`

The number of decimal places to round the data to.

Default: `3`

3.1.3. `int_round`

Rounds a number to a specified precision and returns it as a string.

Parameters

```
int_round(  
    value: number,  
    precision: int  
) -> str
```

value `number`

The number to round.

precision `(int)`

The number of decimal places to round to.

Default: `3`

3.1.4. `is-content-empty`

Checks if a Content is empty (i.e., has no text)

Parameters

`is-content-empty(h: (content)) -> (bool)`

h `(content)`

The content to check.

3.1.5. `is-content-non-empty`

Checks if a content is non-empty (i.e., has text)

Parameters

`is-content-non-empty(h: (content) (str)) -> (bool)`

h `(content)` or `(str)`

The content to check.

3.1.6. `table_from_columns`

Creates a table from a list of columns, as opposed to a list of rows.

Parameters

```
table_from_columns(
  kwargs: arguments (none),
  ..columns: (content)
) -> (content)
```

kwargs `arguments` or `(none)`

Additional keyword arguments to pass to the table function.

..columns `[content]`

The columns to use for the table. Each column should be a list of cells.

3.2. Custom frame utilities

3.2.1. hidden

No-frame style that shows all attributes

Parameters

```
hidden(  
    title,  
    tags,  
    body,  
    supplement,  
    number,  
    arg  
)
```

3.2.2. hidden-no-title

No-frame style that omits the title.

Parameters

```
hidden-no-title(  
    title,  
    tags,  
    body,  
    supplement,  
    number,  
    arg  
)
```

3.2.3. hidden-sup-only

No-frame style that only shows the supplement

Parameters

```
hidden-sup-only(  
    title,  
    tags,  
    body,  
    supplement,  
    number,  
    arg  
)
```

3.2.4. pre-post-style

Frame style that adds pre and post content around the given style.

Parameters

```
pre-post-style(
    pre_content,
    style,
    post_content
)
```

3.2.5. setup-frame-environment

Setup environment for figures using frame-it

Parameters

```
setup-frame-environment(
    doc: [content],
    kind: [str],
    style: [function],
    numbering: [function] [none],
    levels: [int],
    breakable: [bool],
    alignment: [enum]
) -> [none]
```

doc [content]

The document content to configure.

kind [str]

The kind of figure to configure.

style [function]

The frame style to apply. It should have the signature (title, tags, body, supplement, number, arg) → content.

numbering [function] or [none]

The numbering scheme to use for the figure. If none, no numbering is applied.

Default: "1.1"

levels `(int)`

The number of levels to use for dependent numbering. If `numbering` is `none`, this is ignored.

Default: `1`

breakable `(bool)`

Whether the figure should be breakable across pages.

Default: `true`

alignment `(enum)`

The alignment of the figure content. Options are `left`, `center`, `right`.

Default: `center`

3.3. Math symbols

The following shorthands are provided for common math environments:

- `AA`, `BB`, `...`, `ZZ` for the corresponding blackboard bold letters.
- `scr` for script (roundhand) font style. I need this one because my default font has no support for it, so I just fallback to another font.
- `suchthat` for a custom “such that” slash symbol.
- `sfrac`, `hfrac` for skewed and horizontal fractions, respectively.
- Other symbols have also been tweaked.

4. Environments

4.1. Math Environments

The following shows an example usage of the math environments provided by this package.

```
1  #import "@local/vade-mecum:0.1.0": *
2
3  #show: e.set_(
4    settings,
5    heading-numbering: "A.1.",
6    math-numbering: "A.1",
7    math-counter-levels: 2,
8    font-size: 10pt,
9  )
10 #show: template
11
12 = Definitions and notation
13
14 #notation[
15     We write  $\mathbb{N} = \{0, 1, 2, \text{dots}\}$  for the natural numbers,  $\mathbb{Z}$  for the
16     integers,  $\mathbb{Q}$  for the rationals,  $\mathbb{R}$  for the reals and  $\mathbb{C}$  for the
17     complex numbers.
18 ]
19
20 #def[Metric space][
21     A *metric space* is a pair  $(X, d)$  where  $X$  is a set and  $d: X \times X \rightarrow \mathbb{R}$ 
22     satisfies, for all  $x, y, z$  in  $X$ :
23     +  $d(x, y) \geq 0$ , with equality iff  $x = y$ ,
24     +  $d(x, y) = d(y, x)$ ,
25     +  $d(x, z) \leq d(x, y) + d(y, z)$ .
26 ] <def:metric>
27
28 #def[Open ball][
29     For  $x$  in  $X$  and  $r > 0$ , the *open ball* is  $B_r(x) = \{y \in X \mid d(x, y) < r\}$ .
30 ]
31
32 = Results
33
34 #lemma[Balls are open][
35     Every open ball  $B_r(x)$  in a metric space  $(X, d)$  is open.
36 ]
37
38 #proof[[
39     Let  $y \in B_r(x)$  and set  $\epsilon = r - d(x, y) > 0$ . For  $z$  in
40      $B_\epsilon(y)$ , the triangle inequality gives
41      $d(x, z) \leq d(x, y) + d(y, z) < d(x, y) + \epsilon = r$ ,
42     so  $B_\epsilon(y) \subset B_r(x)$ .
```

```

37 ]
38 ] <lem:ball-open>
39
40
41 #theorem[Cauchy--Schwarz][
42   For all  $u, v$  in an inner product space,
43    $|\langle u, v \rangle|^2 \leq \langle u, u \rangle \langle v, v \rangle$ 
44   #proof[[
45     If  $v = 0$  the claim is trivial. Otherwise, for any scalar  $t$ ,
46      $0 \leq \langle u - t v, u - t v \rangle = \langle u, u \rangle - 2t \langle u, v \rangle + t^2 \langle v, v \rangle$ ,
47     and choosing  $t = \frac{\langle u, v \rangle}{\langle v, v \rangle}$  yields the result.
48   ]
49 ] <thm:cs>
50
51
52 == Examples and remarks
53
54 #example[Discrete metric][
55   On any set  $X$ , define  $d(x, y) = 0$  if  $x = y$  and  $d(x, y) = 1$ 
56   otherwise. Then  $(X, d)$  is a metric space in which  $B_1(x) = \{x\}$  for
57   every  $x$ .
58 ]
59
60 #remark[
61   By @lem:ball-open, the open balls form a basis for a topology on  $X$ .
62   Compare this with @def:metric and @thm:cs.
63 ]
64
65 #remark[Calligraphic and blackboard letters][
66   Script letters:  $\mathcal{F}, \mathcal{G}, \mathcal{P}(X)$ . Blackboard letters:  $\mathbb{A},$ 
67    $\mathbb{B}, \mathbb{P}, \mathbb{E}, \mathbb{R}^n$ .
68   Fractions:  $\frac{(1, 2)}{\phantom{x}}$  (skewed) and  $\frac{(1, 2)}{}$  (horizontal).
69 ]

```

D.2. Definitions and notation

Notation: We write $\mathbb{N} = \{0, 1, 2, \dots\}$ for the natural numbers, $\mathbb{Z}$ for the integers, $\mathbb{Q}$ for the rationals, $\mathbb{R}$ for the reals and $\mathbb{C}$ for the complex numbers.

Metric space

Definition D.2.1

A **metric space** is a pair (X, d) where X is a set and $d : X \times X \rightarrow \mathbb{R}$ satisfies, for all $x, y, z \in X$:

1. $d(x, y) \geq 0$, with equality iff $x = y$,
2. $d(x, y) = d(y, x)$,
3. $d(x, z) \leq d(x, y) + d(y, z)$.

Open ball

Definition D.2.2

For $x \in X$ and $r > 0$, the **open ball** is

$$B_r(x) = \left\{ y \in X \mid d(x, y) < r \right\}.$$

D.3. Results

Balls are open

Lemma D.3.1

Every open ball $B_r(x)$ in a metric space (X, d) is open.

Proof

Let $y \in B_r(x)$ and set $\varepsilon = r - d(x, y) > 0$. For $z \in B_\varepsilon(y)$, the triangle inequality gives

$$d(x, z) \leq d(x, y) + d(y, z) < d(x, y) + \varepsilon = r,$$

so $B_\varepsilon(y) \subseteq B_r(x)$. □

Cauchy–Schwarz

Theorem D.3.1

For all u, v in an inner product space,

$$|\langle u, v \rangle|^2 \leq \langle u, u \rangle \langle v, v \rangle.$$

Proof

If $v = 0$ the claim is trivial. Otherwise, for any scalar t ,

$$0 \leq \|u - tv\|^2 = \|u\|^2 - 2t\langle u, v \rangle + t^2\|v\|^2,$$

and choosing $t = (\langle u, v \rangle, \|v\|^2)$ yields the result. □

D.4. Examples and remarks

Discrete metric

Example D.4.1

On any set X , define $d(x, y) = 0$ if $x = y$ and $d(x, y) = 1$ otherwise. Then (X, d) is a metric space in which $B_1(x) = \{x\}$ for every x .

Remark D.4.1: By [Lemma D.3.1](#), the open balls form a basis for a topology on X . Compare this with [Definition D.2.1](#) and [Theorem D.3.1](#).

Remark D.4.2: Script letters: $\mathcal{F}, \mathcal{G}, \mathcal{P}(X)$. Blackboard letters: $\mathbb{A}, \mathbb{B}, \mathbb{P}, \mathbb{E}, \mathbb{R}^n$. Fractions: $(1, 2)$ (skewed) and $(1, 2)$ (horizontal).

4.5. Custom Environments

Custom environments can be defined too.

```
1  #import "@local/vade-mecum:0.1.0": *
2  // ...
3  #let exercise = frame("Exercise", red, kind: "exercise")
4  #show: doc => setup-frame-environment(doc,
5      "exercise", styles.thmbox,
6      numbering: "1", levels: 0, breakable: true,
7  )
8
9  #let solution = frame("Solution", green, kind: "solution")
10 #show: doc => setup-frame-environment(doc,
11     "solution", hidden-sup-only,
12     numbering: none, levels: 0, breakable: true,
13 )
14
15 #exercise[]
16     Given a list of numbers, write a function that returns the sum of the
17     squares of the even numbers in the list.
18
19 #solution[]
20     ```py
21     def sum_of_even_squares(numbers):
22         return sum(number ** 2 for number in numbers if number % 2 == 0)
23     ```
24 ]
```

Output:

Exercise 1

Given a list of numbers, write a function that returns the sum of the squares of the even numbers in the list.

Solution:

```
1  def sum_of_even_squares(numbers):
2      return sum(number ** 2 for number in numbers if number % 2 == 0)
```